Artificial Intelligence Review (2025) 58:374
https://doi.org/10.1007/s10462-025-11358-9

CTWA: a novel incremental deep learning-based intrusion detection method for the Internet of Things

Haizhen Wang[1,2] · **Yutong Yang**[1,2] · **Pan Tan**[1,2]

Accepted: 14 August 2025 / Published online: 8 October 2025
© The Author(s) 2025

Abstract
Class incremental learning aims to learn new courses in an incremental manner without forgetting the categories previously learned. A novel incremental Internet of Things (IoT) intrusion detection method CTWA based on Convolutional Autoencoder (CAE) and Temporal Convolutional Network (TCN) is proposed to address the issues of insufficient generalization ability, high computational resources, and redundant features in class incremental learning. This method first completes the training of the CAE-TCN module, extracts and concatenates local features of data samples through CAE and TCN, and then initializes the incremental learning module. Residual modules are added to the CAE to improve the training effect of the model and avoid gradient vanishing problems. The CAE-TCN module shares lower-level feature representations through task-specific layers in incremental learning module. It distinguishes between old and new tasks using Gaussian distribution, and applies Weight alignment (WA) techniques between task heads to ensure that learning the new task does not result in forgetting the knowledge of the old tasks. Ultimately, the outputs of both new and old tasks are weighted and fused to ensure the optimal classification result. Additionally, a loss function combining cross-entropy loss and label smoothing loss is used to enhance the model's performance. We conducted experiments on two datasets. The experimental results on CICIoT2023 dataset demonstrate that the proposed model excels in terms of accuracy, precision, recall, and F1-Score, achieving 0.9643, 0.9659, 0.9643, and 0.9645, respectively. With 40 training epochs, the model's runtime is 789.58 s, which is higher than most comparison models, but the accuracy is significantly improved. The proposed method can effectively distinguishes between different known and unknown types of attacks, highlighting its potential applications in the field of cybersecurity.

Keywords Convolutional autoencoder · Temporal convolutional network · Incremental learning · Weight alignment · Deep learning

✉ Haizhen Wang
wanghaizhen1976@163.com

[1] College of Computer and Control Engineering, Qiqihar University, Qiqihar 161006, China

[2] Heilongjiang Key Laboratory of Big Data Network Security Detection and Analysis, Qiqihar University, Qiqihar 161006, China

1 Introduction

There are numerous protocols and no unified standards in Internet of Things (IoT), which face many security risks and are easily exploited by hackers for activities such as access control threats and zero-day attacks. Therefore, network security is an important prerequisite for IoT to be able to develop rapidly, and IoT intrusion detection systems are designed to monitor, detect and respond to attempts to compromise the integrity and confidentiality of resources in IoT systems. Currently, many deep learning-based detection methods have been proposed one after another to cope with IoT security issues, such as Gated Recurrent Unit (GRU) (Xu et al. 2018) and Deep Belief Networks (DBNs) (Lopes et al. 2015). However, these methods do not cope well with the dynamic nature in IoT traffic data with fully accessible default datasets and known dataset categories, which makes it difficult for already trained models to accurately identify traffic types when new attacks emerge. The attack categories of IoT traffic are endless, and if the model is retrained after new data is available, it will put higher demands on the computational power and storage of IoT devices. In incremental learning, as the data flow is updated, the intrusion detection model's ability to recognize the data flow is gradually enhanced, thus ensuring real-time dynamic data flow detection.

Incremental learning (IL) is the ability of Artificial intelligence (AI) to learn from data presented to them sequentially. Our focus is on the IL class, which assumes that the data is labelled. This problem is trivial if sufficient computational power and storage are available and long delays are allowed for model updates. These conditions are usually not met in real applications and the IL class becomes difficult to solve.

In the Class incremental learning (CIL) scenario, deep neural network models are not trained offline on a fixed pre-collected dataset, but sequentially on new incoming data. The model must be incrementally updated using only a limited number of new classes at a time, with the goal of learning a uniform classifier across all visible classes. This paradigm is similar to the human learning process and is challenging because models are suffering from catastrophic forgetting: learning new data or classes will result in a severe degradation of past performance. Several studies have emphasized the importance of rehearsal for continuous and incremental learning. Our contribution are as follows:

1. Introduction of residual module: we add a residual module to CAE to increase the model depth by using jump connections, which effectively solves the problem of gradient vanishing and improves the training speed so as to extract features more comprehensively.
2. Optimized Loss Function: During the optimization of model parameters, we employ a loss function that integrates cross-entropy loss with label smoothing loss. This approach balances the classification boundary, reduces the model's over-reliance on labels, and maintains the efficiency and intuitiveness of cross-entropy loss. Additionally, by introducing label smoothing, the model's generalization ability and robustness have been enhanced.
3. Temporal and spatial feature extraction: we extract spatial features by CAE and time series features by TCN, splicing their outputs to form a CAE-TCN module in order to comprehensively extract the spatial and temporal series features of the dataset, thus improving the accuracy of the model.

4. In the process of incremental learning, we utilize task-specific layers to share lower-level feature representations and use a Gaussian distribution to distinguish between new and old tasks. By applying WA techniques between task heads, we ensure that the learning of new tasks does not lead to the forgetting of knowledge from old tasks. Ultimately, by weighted fusion of the outputs from both new and old tasks, we ensure the optimal classification results and can effectively handle cases where the feature differences of new categories are significant, successfully learning the new categories.
5. We conducted experiments on the proposed model on datasets CICIoT2023 and BoT-NetIoT, the experimental results show that our method outperforms other models in terms of accuracy precision, recall, and F1 score.

The remaining arrangements of the paper are as follows. Section 2 provides a literature review on incremental learning based on deep learning. Section 3 provides a detailed introduction to our proposed incremental learning intrusion detection framework and introduces the dataset and related technologies used in the framework. Section 4 introduces the evaluation methods adopted and the analysis of the results of the proposed model. Finally, Sect. 5 summarizes some of the applications discussed in this paper and the limitations of the proposed model, thereby proposing some future research areas.

2 Relevant research

Existing class incremental learning methods fall into three main categories:

Class incremental learning based on knowledge distillation: such algorithms require the retention of the previous stage of the model during model updating, resulting in the need to forward compute the same input sample twice, which significantly increases training and storage overheads. For example, the Learning without Forgetting (LwF) (Li and Hoiem 2017) and Incremental Classifier and Representation Learning (iCaRL) (Rebuffi et al. 2017) methods are intuitively constrained and easy to implement, but also introduce additional complexity.

Class incremental learning based on hysteresis adjustment: In the process of mitigating model bias, such algorithms often introduce additional statistical information, which can increase operational complexity and storage overhead. For example, methods like BiC (Wu et al. 2019) and WA effectively reduce model bias.

Parameter regularity-based class incremental learning: such algorithms need to store parameter importance matrices of the same size as the model, leading to significantly higher storage overheads in streaming data learning, e.g., the Elastic Weight Consolidation (EWC) (Kirkpatrick et al. 2017) method need to store constraint matrices and thus have higher overheads.

In addition, class-based incremental learning algorithms based on dynamic model architectures need to continuously increase the model size during the model update process, which again leads to a gradual increase in storage overheads, such as the Dynamically Expandable Networks (DEN) (Lopez-Paz and Ranzato 2017) method, whose model parameters grow linearly with the data. Data replay-based class incremental learning algorithms, on the other

hand, rely on the process of reviewing a small sample set and are susceptible to overfitting and data imbalance, e.g., the Generative Replay (GR) (Yoon et al. 2017) method, although easy to manipulate, tend to lead to overfitting. In contrast, class incremental learning algorithms based on data constraints treat the samples as indicators, assuming that the model's performance on the old class will not deteriorate as long as the model is guaranteed not to increase in loss on the example set, e.g., the Gradient Episodic Memory (GEM) (Shin et al. 2017) method, but the optimization complexity is high and relies on assumptions.

In the CAE-TCN model, CAE and TCN are used as feature extractors to extract spatiotemporal features of the samples. Although CAE is used for feature extraction, it can map the input data to a low-dimensional potential space, reducing the number of parameters by sharing the weights of the convolutional layers while preserving the local structural information of the spatial features. This makes the model computationally efficient and suitable for large-scale network intrusion detection environments. eTag, a new approach that does not require the storage of examples or prototypes, was proposed by Huang et al. (2023) to address the well-known forgetting problem of networks whose performance degrades drastically when the task is updated, by embedding distillation and task-directed generation that prevents the feature extractor from forgetting based on the lack of the need for examples or prototypes, while enabling a lightweight generator to produce dynamic features for top incremental classifiers. Zhang et al. (2023) proposed an open-set identification CIL algorithm ADCIL based on an anomaly detection mechanism, which employs anomaly detection to differentiate between tasks, learns valid open-set knowledge through its own training set, and maintains the old knowledge using knowledge distillation or fine-tuning constraints. By enhancing the ability to distinguish between tasks in the network model, the robustness of the algorithm is effectively improved and the phenomenon of catastrophic forgetting is prevented. Liu et al. (2021) proposed a Reinforcement Memory Management (RMM) strategy for class-incremental learning, which optimizes the dynamic memory management through reinforcement learning and divides the memory allocation strategy into two levels, namely, the division between old and new categories and the allocation for each specific category, which significantly improves the performance in the class incremental learning setting. Petit et al. (2023) proposed a method called FeTrIL, which effectively improves the performance of class incremental learning without exemplars by combining a fixed feature extractor and a pseudo-feature generator in order to balance the stability and plasticity, and outperforms the existing methods in a number of experiments. Dong et al. (2022) proposed a model called GLFC to solve the severe forgetting problem of global models due to time change in federated learning. The local forgetting of local clients is balanced by designing the class perceptual gradient compensation loss and the class semantic relation distillation loss, and the global forgetting caused by non-independent same-distribution category imbalance across clients is resolved by selecting the best old global model to assist the local relation distillation. Yan et al. (2021) proposed a novel two-phase learning method in order to solve the catastrophic forgetting problem, which exploits dynamically scalable representations for more effective incremental concept modelling. Specifically, in each incremental step, we freeze the previously learnt representation and augment it with additional feature dimensions from a new learnable feature extractor. Zhu et al. (2021b) To address the catastrophic forgetting problem, propose a simple non-sample-based approach called PASS. In this method, each previously learned class stores a prototype representation and utilizes prototype augmentation within the deep feature space. This helps preserve

knowledge of past classes without relying on stored samples. Van De Ven et al. (2021) argue that most existing methods for incremental class learning store data or use generative replay, both of which have drawbacks, and that "unrehearsed" alternatives such as parameter regularization or bias correction methods do not always achieve high performance. Therefore, a new strategy for incremental class learning is proposed: using a joint distribution for learning, followed by classification using Bayes' rule, training a variational autoencoder for each class to be learnt and estimating the likelihood p (x | y) using importance sampling to implement the strategy. solves the problem of rapid scaling of dynamic models as the number of tasks increases. These connections enable knowledge to be transferred from old tasks to new ones through feature sharing and reuse, but their number of references is large and model scaling is complex. Hu et al. (2023) proposed a novel NE method, Dense Network Expansion (DNE), aimed at better balancing accuracy and model complexity. This method introduces dense connections in the intermediate layers of task-specific expert networks, enabling efficient knowledge transfer from previous tasks to new ones through feature sharing and reuse. Such sharing is achieved via a cross-task attention mechanism, based on a newly designed Task Attention Block (TAB), which facilitates information fusion across tasks. Unlike traditional attention mechanisms, TAB operates at the feature mixing level and is decoupled from spatial attention, making it more suitable for CIL than joint spatial and task attention. DNE effectively maintains the feature space of previous classes while expanding the network and feature scale at a significantly slower rate. Jodelet et al. (2023) address the problem of using additional data to mitigate catastrophic forgetting by proposing an approach that utilizes a pre-trained stable diffusion model as an additional data source for incremental learning of categories. The method not only uses these extra data samples in distillation loss, but also replays them in categorical loss. To address the catastrophic forgetting problem, Zhu et al. (2021a) proposed explicit class augmentation and implicit semantic augmentation to address both representation bias and classifier bias without the need to store old samples. The FOSTER method introduced by Wang et al. (2022) enables the model to adaptively learn new categories by dynamically expanding the new module while retaining the feature descriptors of the previously learned categories. Tabassum et al. (2021) proposed a new privacy-preserving distributed incremental learning intrusion detection pipeline to address the problem that existing techniques for incremental learning are computationally expensive and produce duplicate features. The method optimizes load distribution for feature extraction in IoT and edge devices by reducing redundancy and employing autoencoders with non-negative constraints. Finally, to improve the detection of cyber-attacks, Zhao et al. (Zhao et al. 2022). proposed an improved model combining Temporal Convolutional Networks (TCNs) and attention mechanisms. The model leverages causal and dilation convolutions to capture the spatiotemporal dependencies in the data, while cross-layer information transfer is achieved through residual module, enhancing the deep network's learning capability. Additionally, an attention mechanism is employed to improve the model's focus on anomalous features associated with various attacks (Huang et al. 2023; Dong et al. 2022) .

Insufficient generalization ability, when dealing with complex category changes and diverse data distributions, some of the methods show insufficient generalization ability to adapt to new categories or tasks (Liu et al. 2021; Hu et al. 2023; Wang et al. 2022). High computational resources, the methods require a lot of computational resources and time, especially when learning new categories or sharing knowledge between tasks, which limits

their application in resource-constrained environments (Petit et al. 2023; Zhu et al. 2021b; Ven et al. 2021). Increased model complexity, when dealing with new tasks or categories, some of the methods are progressively less efficient as the model complexity increases, which affects their performance in incremental learning (Zhang et al. 2023). Knowledge distillation and fine-tuning constraints are limited and may not be effective in retaining knowledge from old tasks, especially when prone to model collapse under large-scale data (Yan et al. 2021; Tabassum et al. 2021). Feature redundancy and local feature neglect, some methods over-rely on global features and neglect local anomalous features, leading to feature redundancy and affecting model performance on complex datasets (Jodelet et al. 2023; Zhao et al. 2022). Dependence on external resources, some methods rely on external resources such as pre-trained models or diffusion models, and the model performance is susceptible to the quality of external resources, which may introduce bias.

Therefore, inspired by Zahoora et al. (2022) and Ashraf et al. (2019), we proposed a novel incremental IoT intrusion detection method, which can effectively distinguish between different known types of attacks. Also, our method can detect zero-day attacks by identifying unknown types of attacks, as follows: First, the CAE is used to extract the structural and behavioral features of ransomware, enabling the learning of robust representations from raw data to more accurately capture its core behavioral patterns. The TCN effectively captures local temporal features during the execution process through stacked convolutional structures and models a certain degree of long-term dependencies. In conjunction with the static features extracted by the CAE, this facilitates the mapping of complex low-level variations into a unified and semantically expressive feature space. On this basis, a task-specific layer sharing mechanism is introduced, and a Gaussian distribution is employed to model the fine-grained structure within the semantic feature space, helping to characterize the typical feature distribution of known ransomware samples. By computing the likelihood of input samples under the distribution of previous tasks and comparing it with that of new tasks, and by incorporating a composite loss function and an incremental training mechanism, our method is able to continuously learn emerging attack behaviors, thereby enabling the detection of zero-day attacks.

In summary, our method combines CAE and TCN to effectively extract spatiotemporal features of the dataset, effectively reduce the redundant feature dependency, and ensure the efficient use of data information. We use a loss function combining cross-entropy loss and label smoothing loss to balance the classification boundary and reduce the model's over-reliance on labels. In the incremental learning module ensure that learning new tasks does not forget the knowledge of old tasks. Finally, the outputs of new and old tasks are fused with weights, maintaining the performance of original tasks while reducing excessive computational resource consumption.

3 Modelling

3.1 Modelling framework

This model uses a combination of CAE and TCN to effectively extract local features from data samples. Next, task-specific layers are responsible for sharing lower-level feature representations. Gaussian distribution is used to determine whether a sample belongs to a new

task, with the goal of distinguishing between new and old tasks. Based on this determination, samples are sent to different task heads for processing. WA techniques are applied between task heads to ensure that learning new tasks does not forget the knowledge of old tasks. Finally, the outputs of both new and old tasks are weighted and fused to ensure optimal classification results. Additionally, combining cross-entropy loss with label smoothing loss helps improve model performance and prevent overfitting. The classification task is completed using the Softmax function. The model structure is shown in detail in Fig. 1.

In the model M stands for old categories and N stands for new categories. WA stands for Weight Alignment Approach.

Data preprocessing includes data normalization and numerical processing (see Sect. 3.2.1 for details). A residual module is introduced in CAE to enhance feature representation. The training model performs parallel feature extraction by CAE and TCN followed by feature fusion. The bottom-level feature representations are shared through task-specific layers, with Gaussian distribution used to distinguish between new and old tasks. WA techniques are applied between task heads to ensure that learning new tasks does not lead to forgetting knowledge from old tasks. Finally, the outputs of new and old tasks are weighted and fused to enhance the overall model performance. The loss function combines label-smoothing loss and cross-entropy loss, aiming to improve the model's ability to generalize during incremental learning, mitigate overfitting problems, and prevent catastrophic forgetting. The effectiveness of the model in dynamic environments is ensured by improving the robustness and stability of the model to new classes of data.

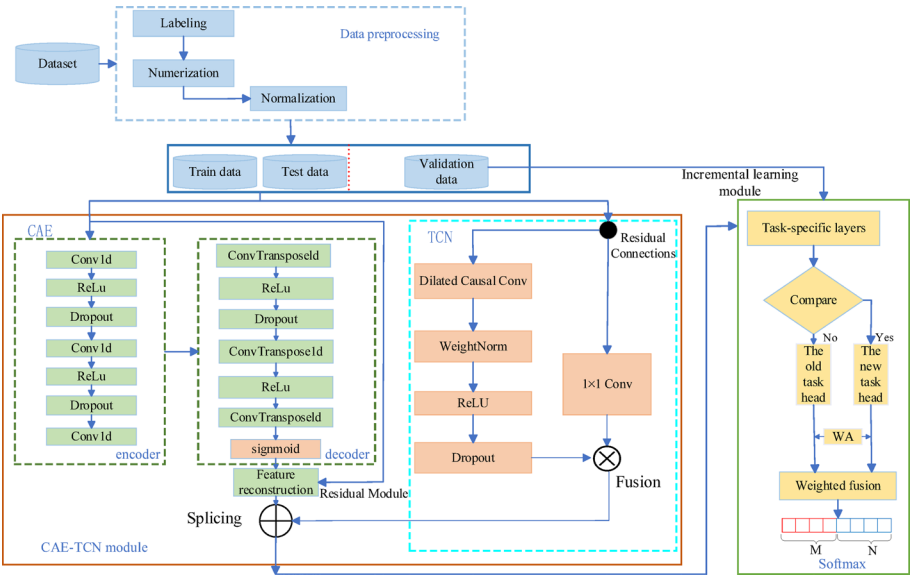


Fig. 1 CTWA model framework

Table 1 Distribution of the CICIoT2023 dataset

Dataset	Tab	Typology	Quantities	Tab	Typology	Quantities
CICIoT2023	0	Benign	10,117	1	DDoS-ACK	4680
	2	DNS	5790	3	DDoS-ICMP	5243
	4	DDoS-HTTP	5202	5	DDoS-SYN	7344
	6	DDoS-TCP	10,447	7	DDoS-UDP	4704

Table 2 Distribution of BoTNeTIoT dataset

Dataset	Tab	Typology	Quantities	Tab	Typology	Quantities
BoTNeTIoT	0	Ack	14,700	1	Combo	14,004
	2	Junk	14,279	3	Scan	14,352
	5	syn	13,266	5	TCP	14,362
	6	UDP	14,498	7	UDPPLAIN	14,509

3.2 Datasets and preprocessing

3.2.1 Datasets

In our model we use the CICIoT2023 dataset, which contains most of the attack types and is currently the most widely used dataset for future intrusion detection. The CICIoT2023 dataset involves 232,885 connections with 47 features including 33 sub-attacks classified into 7 different attack classifications; DoS, DDoS, Web-based attacks, Recon, Spoofing, Brute force, and Mirai (Neto et al. 2023). In our experiments, we removed only the timestamp feature, as it does not reflect network behavior and is primarily used for ranking purposes. Other features are directly used to evaluate the performance of different machine learning models in this setting, with a total of 46-dimensional features.

In this paper, the CICIoT2023 dataset is used to train the model, and the categories include Benign, DDoS-ACK, DNS, and DDoS-ICMP. Four categories, DDoS-HTTP, DDoS-SYN, DDoS-TCP, and DDoS-UDP are added to validate the incremental learning performance. We will show the specific sample distribution of the incremental learning module through Table 1 in Sect. 4.2. In addition, our experiments use the BoTNetIoT (Koroniotis et al. 2019) dataset, which contains traffic from nine devices and where Mirai botnet attack is the only attack types in the BoTNetIoT dataset. The dataset consists of 23 features extracted from five different time windows. The exact distribution of the selected samples will also be shown in Table 2 of Sect. 4.2.

3.2.2 Data preprocessing

Before conducting the experiments, we performed data preprocessing including numerical encoding, normalization, and standardization. The dataset was then split into a training set and a testing set with a ratio of 7:3. The incremental part used a validation set, which included new, previously unseen attack samples. The detailed processing steps are as follows.

(1) Labeling: In Labeling, each category of a categorical variable is mapped to a unique integer label so that the model can handle it.

(2) Numerization: In data preprocessing, numerical encoding refers to the process of converting non-numeric data (such as text or categorical data) into a numerical format that can be accepted by the model.
(3) Normalization: In the final stage of preprocessing, the data are normalized. Use Eq. (1) to scale the data into [0,1].

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{1}$$

x is the sample data,x' is the normalized data, $x_{\max}$ is the maximum value of the sample data and $x_{\min}$is the minimum value of the sample data.

3.3 Convolutional autoencoder

CAE is a autoencoder based on Convolutional Neural Networks (CNNs) that combines the spatial feature extraction capability of CNNs and the unsupervised learning characteristics of autoencoders to efficiently learn compact representations of data. CAE encodes and decodes data by means of a convolutional layer and a pooling layer, which is particularly suitable for structured data and extracting spatial hierarchical features.

Convolutional layer-Convolution: Initialize k convolutional kernels (w), each with a bias b , and convolve them with the input x to generate k feature maps h , as shown in Eq. (2).

$$h^k = \sigma(x * w^k + b^k) \tag{2}$$

Convolutional layer autoencoder (deconvolution operation): Each feature map h is convolved with the transpose of its corresponding convolution kernel and the result is summed, then biased with c, as shown in Eq. (3).

$$\mathrm{y} = \sigma(h^k * \hat{w}^k + c) \tag{3}$$

Convolutional Layer - updating Weights: To update weights, the first step is to determine the loss function. The MSE (Mean Squared Error) function is used to subtract the sum of squares of the target value and the actual value, and then calculate the mean. As shown in Eq. (4).

$$\mathrm{E} = \frac{1}{\mathrm{n}} \sum (x_i - y_i)^2 \tag{4}$$

Traditional autoencoders rely on fully connected layers, have a large number of parameters, and are difficult to capture the spatial structure of the data. CAE, on the other hand, utilizes convolution and pooling operations, which significantly reduces the number of parameters while being able to preserve the spatial features of the data, and its convolution operations are performed only in local regions with lower computational complexity. This paper combines CAE with TCN with the aim of extracting more features to improve the accuracy of the model. At the same time, CAE introduces a residual module to compensate for data lost

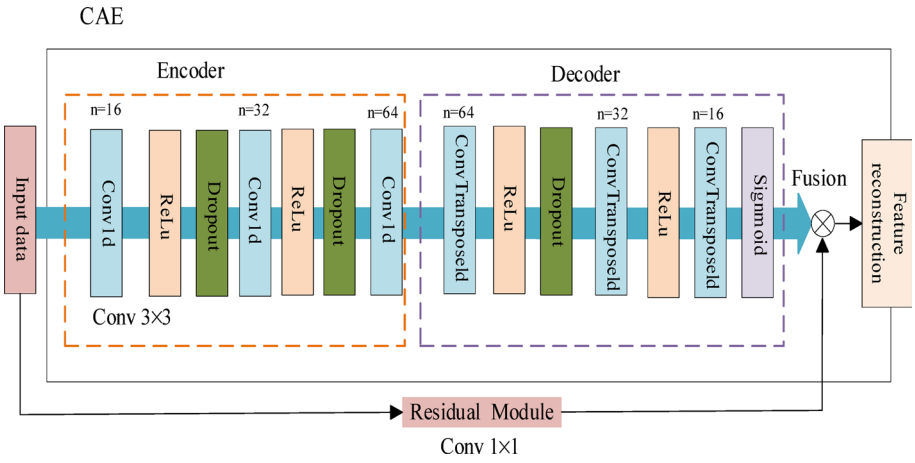


Fig. 2 CAE network structure diagram

during decoding and to address the problem of gradient vanishing. By combining with TCN, the model is able to extract the temporal features of the samples while maintaining the spatial features, thus enhancing the training effect of the model. Here, n represents the number of convolutional kernels. The CAE network structure of this paper is shown in Fig. 2.

The output network is designed as $H(x) = F(x) + x$, the residuals module is added to the result of x by convolution 1×1, and the result of CAE by sigmoid is $F(x)$, and finally, the "sum" operation is performed to compensate for the data lost in the decoding process and to solve the problem of vanishing gradient.

3.4 Temporal convolutional networks

Causal convolution performs convolutional computations on data from past time steps only in strict temporal order, ensuring that the values in the upper time step t depend only on the values in the lower layer at time step t and before. Causal convolution cannot see future data; it is a unidirectional structure. In causal convolution, the definition of the time step t ensures that the output value at time step t depends only on the current and previous time steps. As shown in Eq. (5).

$$y_t = \sum_{k=0}^{K-1} w_k \cdot x_{t-k} + b, \tag{5}$$

where y_t is the value of the output sequence at time step t, $w_k$is the weight of the convolution kernel at position k, $x_{t-k}$is the value of the input sequence at time step t−k, k is the size of the convolution kernel, and b is the bias term.

Causal convolution uses dilation convolution to achieve exponentially large sensory fields, and the dilation convolution operation F on element s in the sequence is shown in Eq. (6).

$$F(s) = (x_d^* f)(s) = \sum_{i=0}^{k-1} f(i) \cdot x_{s-d \cdot i} \tag{6}$$

Expansion convolution adds an "expansion rate" parameter compared to traditional convolution, which effectively enhances feature extraction without increasing the number of parameters, allowing the model to expand the perceptual range of the convolution kernel. The dilation convolution skips parts of the input with hyperparameters. As a result, the filter can operate over a larger range than itself. When combined with causal convolution, for a one-dimensional input l and a filter$f : \{0, 1, \ldots, k-1\}$, the expansion convolution in layer r is calculated using Eqs. (7) and (8).

$$\mathrm{F}(l_t) = (l_{d_r}^* f)(t) = \sum_{i=0}^{k-1} f_j l_{t-d_r j} \tag{7}$$

$$sequence = (F(l_1), F(l_2), \ldots, F(l_T)), \tag{8}$$

where $F(\cdot)$denotes the convolution operation, k denotes the convolution kernel size, and d_r denotes the expansion factor of the r-layer.

The residual module is modelled as shown in Fig. 3. Each residual module has two dilation convolution layers, a weight normalization layer, and an activation function of ReLu, in addition, the TCN achieves regularization by adding dropout to each residual module after the inflated convolution. Considering the potential variability in the number of input and output channels, a 1×1 convolutional layer is used to adjust the number of channels. Its output is added to the input of the module x : $o = Activation(x + F(x))$.

The structure of the TCN used in this paper is shown in Fig. 3.

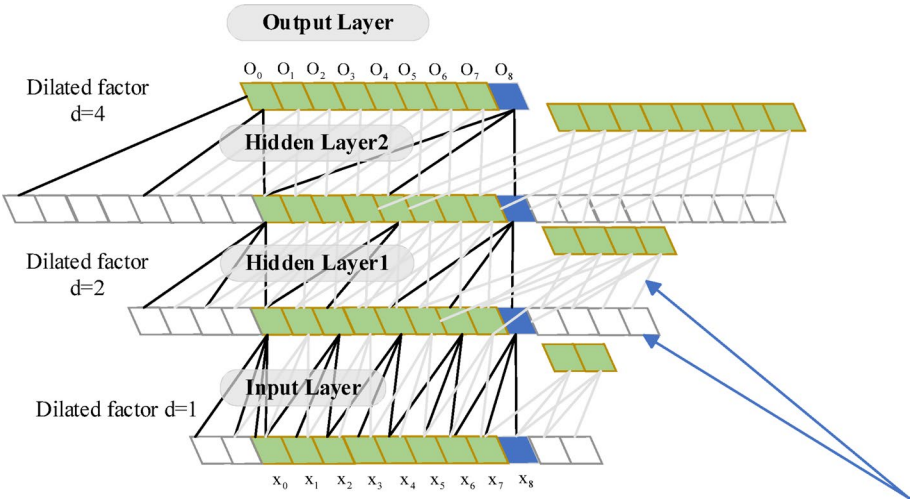


Fig. 3 TCN structure diagram

Here, a nine-element sequence [x0, x1, x2, x3, x4, x5, x6, x7, x8] is taken as an example, and the residual module with an initial convolution kernel of 4 is followed by the expansion coefficients of {1, 2, 4}, respectively, and padding thereafter. From a micro-computational point of view, the computational process and data flow of the residual module is presented layer by layer from bottom to top as shown in Fig. 3. The green squares in the figure are the valid data, coming from the input data or the output of each layer of residual module. The white squares are filler data. The line in Fig. 3 indicates the convolution operation. The module line indicates the relationship between the last element of the output vector and all the elements of the TCN involved in the process. As the calculation moves up the chart, the actual information will gradually move to the right. The nth element of the output sequence is computed from the first n elements of the input elements. The nth aspect of the output sequence is only affected by the first n elements of the input sequence, forming a causal relationship.

3.5 Loss function

In this paper, we introduce the cross-entropy loss function and a loss function that combines with the label smoothing loss to significantly improve the performance of the model while coping with problems such as category imbalance and overfitting. Firstly, review the cross-entropy loss function, as shown in Eq. (9).

$$\mathrm{L} = \frac{1}{\mathrm{N}} \sum L_i = -\frac{1}{N} \sum_i \sum_{c=1}^{M} y_{ic} log(p_{ic}) \tag{9}$$

M represents the category: and i denotes the ith sample.y_{ic} is the label (0 or 1) for the category c of the ith sample, and p_{ic} is the probability of the ith sample belonging to category c.

The cross-entropy loss function after label smoothing is shown in Eq. (10).

$$\begin{aligned} H(q', p) &= -\sum_{k=1}^{k} \log p(k) q'(k) \\ &= (1-\varepsilon) H(q, p) + \varepsilon H(u, p) \end{aligned} \tag{10}$$

where $H(q, p)$ is the cross-entropy function, q is the labelled value, $p$is the predicted outcome, and k is the category. The label smoothing of q is changed to q', so that the distribution of p output by the model to approximate q'. u(k) is a probability distribution, here the uniform distribution is used $u(k) = \frac{1}{k}$, where εprobability comes from the uniform distribution, $1 - \varepsilon$probability comes from the original distribution.

3.6 Incremental learning module

In this paper, the task-specific layers in incremental learning are responsible for sharing lower-level feature representations. By using a Gaussian distribution to determine whether a sample belongs to a new task, the distinction between new and old tasks is achieved. By using Gaussian distribution to determine whether a sample belongs to a new task, we can

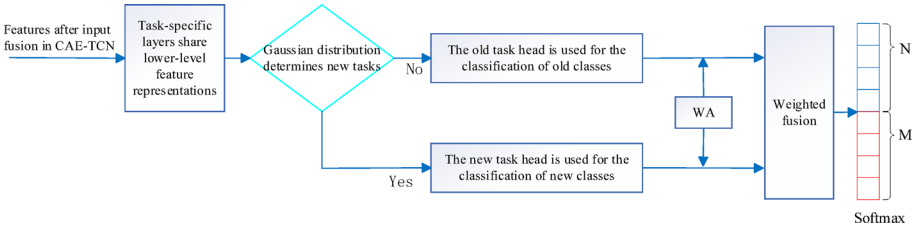


Fig. 4 Flow chart of incremental learning module

distinguish between old and new tasks. Based on this judgment, the sample is routed to different task heads for processing. WA techniques are applied between the task heads to ensure that learning the new task does not result in forgetting the knowledge from the old tasks. Finally, the outputs of the new and old tasks are weighted and fused. This approach allows the model to learn new category features while retaining the ability to extract features from old categories, thereby enabling effective incremental learning. The incremental learning module of this paper is shown in Fig. 4.

4 Analysis of experimental results

In this paper, widely used evaluation metrics are used to assess the performance of classification models. These metrics are Accuracy, Precision, Recall and F1 Score, as shown in Eqs. (11)–(14).

$$Accuracy = \frac{(TP + TN)}{(TP + TN + FP + FN)} \tag{11}$$

$$Precision = \frac{(TP)}{(TP + TN + FP + FN)} \tag{12}$$

$$Recall = \frac{(TP)}{(TP + FN)} \tag{13}$$

$$F_1\text{-}score = \frac{(2 * TP)}{2 * TP + FP + FN} \tag{14}$$

where *TP*, *TN*, *FP*, *FN* stands for True Positive, True Negative, False Positive, False Negative respectively.

4.1 Experimental environment

This experiment was implemented using Visual Studio code development environment under Intel(R) Core (TM) i5-12600KF 3.70 GHz, 32G RAM, 64-bit hardware environment and Windows 10 operating system, and programmed through Python 3.10.14 under PyTorch framework.

Table 3 CAE-TCN convolutional layer structure

Convolutional layer	layer name	Convolution kernel scale, step size, number of output channels
CAE	Conv1	1,16,7
	Conv2	16,32,7
	Conv3	32,64,7
	ConvTranspose1d	32,64,7
	ConvTranspose1d	16,32,7
	ConvTranspose1d	1,16,7
Residual	Conv7	1,1,1
TCN	Conv1	1,32,3
	Conv2	32,64,3

Table 4 Effect of convolutional kernel sizes of CAE performance on CICIoT2023 dataset

Model	ACC	Precision	Recall	F1-score	Loss
Kernel size = 2	0.985	0.987	0.985	0.985	0.047
Kernel size = 3	0.988	0.989	0.988	0.988	0.038
Kernel size = 4	0.988	0.988	0.988	0.986	0.040
Kernel size = 5	0.990	0.995	0.990	0.990	0.034
Kernel size = 7	0.993	0.994	0.993	0.992	0.028

4.2 CAE-TCN model data and distribution

In this paper, in order to better validate the effect of data generation, two datasets are used in this paper for validation, and the distribution of the used datasets is shown in Tables 1 and 2.

4.3 Parametric experiments

4.3.1 CAE-TCN parameter settings

The CAE model uses six convolutional layers and the structure of the convolutional layers for CAE and TCN is shown in Table 3.

4.3.2 Impact of convolutional kernel sizes on model performance

The number and size of convolutional kernels have an important impact on model performance, so it is crucial to choose the right convolutional kernel. It has been verified through a large number of experiments that increasing the number of convolutional kernels increases the number of parameters and computation of the model, which may lead to overfitting and prolong the training time, while not necessarily improving the performance and recognition rate of the model. On the contrary, an appropriate number of convolutional kernels can extract features more efficiently and reduce the model complexity, thus improving the model's generalization ability and operation efficiency. In this paper, we set the number of convolution kernel sizes as {2,3,4,5,7} for experiments on CICIoT2023 dataset, and the experimental results are shown in Table 4, and the accuracy and loss rates are shown in Figs. 5 and 6.

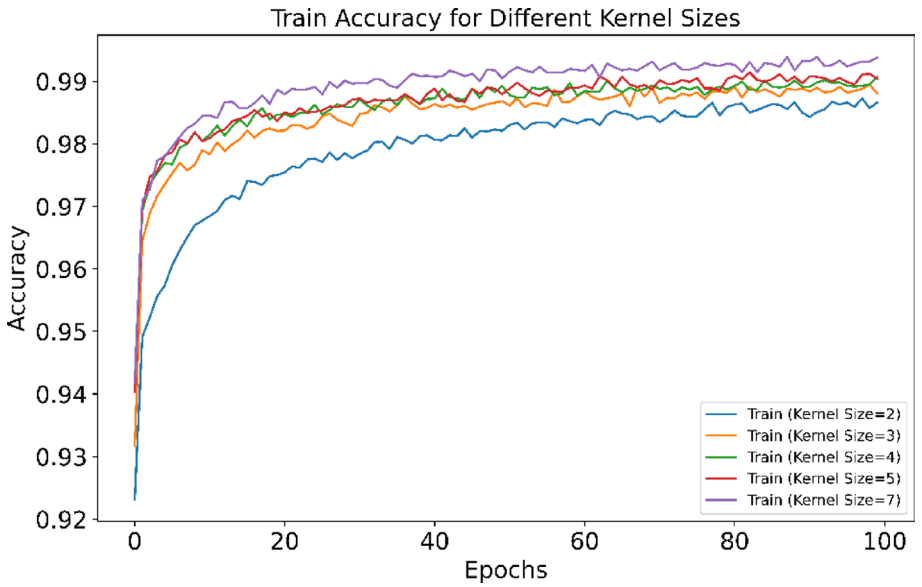


Fig. 5 Training accuracy for different convolutional kernel sizes on CICIoT2023 dataset

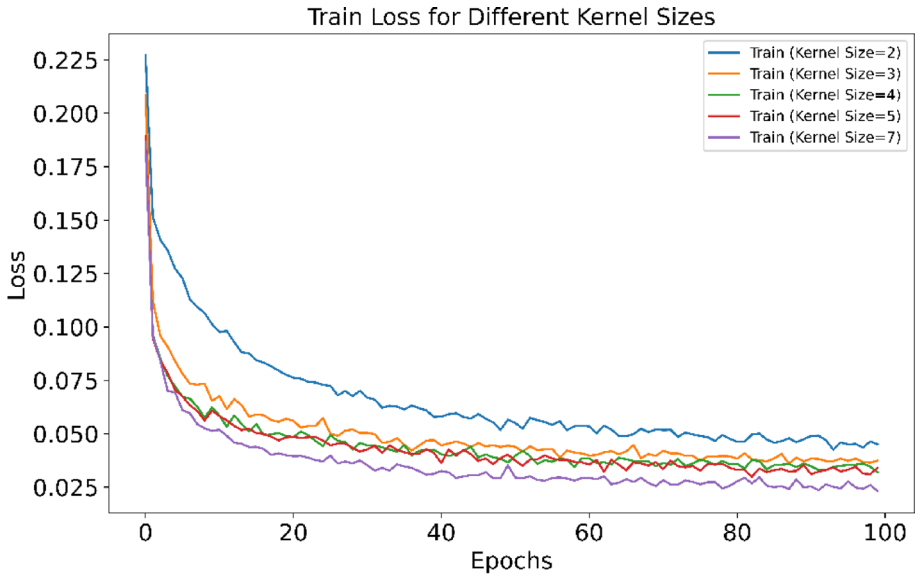


Fig. 6 Training loss for different convolutional kernel sizes on CICIoT2023 dataset

As shown in Table 4; Figs. 5 and 6, when the convolutional kernel size is set to 7, the model achieves slightly higher validation accuracy compared to other convolutional kernel sizes, while the validation loss is lower than with the other settings. Therefore, this paper ultimately selects a convolutional kernel size of 7.

4.4 Ablation experiments

This section analyses the contribution of different modules of the CAE-TCN model to the overall performance through ablation experiments, specifically CAE-TCN, TCN-WA, CAE-TCN-Cross Entropy, CAE-TCN-Label-Smoothing. The results of the ablation experiments on CICIoT2023 dataset are shown in Table 5, which show that the accuracy of this paper's model is 6.13% points higher than that of CAE-WA, 0.61% points higher than that of TCN-WA, 0.11% points higher than that of CAE-TCN-Cross Entropy-WA, and 0.15% points higher than that of CAE-TCN-Labelling-Smoothing-WA, Although the accuracy of the model has been significantly improved, the running time has also increased accordingly. However, from the perspective of time efficiency, the running time of this model is still better than other similar models.

The datasets used in this paper is relatively balanced with high-quality labels, which means that the cross-entropy loss can effectively utilize these clear label signals to train the model. In this case, introducing label smoothing introduces unnecessary noise, weakening the model's learning signal for the correct class and thus reducing its performance. The combined use of label smoothing loss and cross-entropy loss yields better results than using either alone because they complement each other: cross-entropy loss ensures the model has sufficient discrimination power for the correct class, while label smoothing loss regularizes the model to prevent overfitting and enhance generalization. This combination demonstrates stronger robustness when facing label noise or overfitting issues. By adjusting the weights of both losses and the smoothing parameter, an optimal configuration can be found for the dataset.

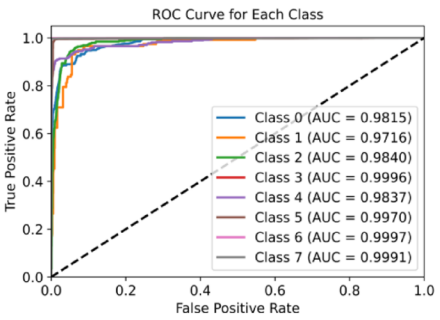
The ROC and confusion matrices for the ablation experiments are shown in Figs. 7 and 8.

4.5 Comparative experiments

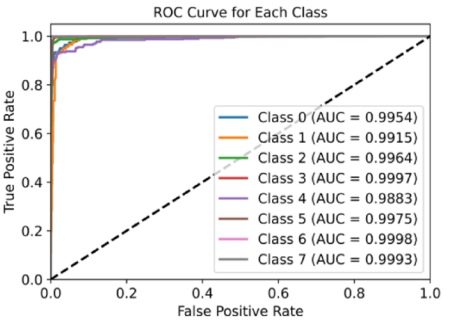
In order to verify the overall performance of the models in this paper, SSAE-SOINN (Liu et al. 2023), BDPT (Wen and Zhu 2023), SOINN (Wu et al. 2023), IPCA-CNN (Wang et al. 2023), and NIDS (Tian et al. 2023), which have excellent performance in incremental learning, are used for comparison. The time module of python was used to calculate the model time. In the CICIoT2023 dataset, four categories were selected for training, followed by four new categories for incremental learning validation. The results of the comparative experiments are shown in Table 6, demonstrating that the accuracy of the proposed model is 8.53% points higher than SSAE-SOINN, 3.57% points higher than BDPT, 12.50% points

Table 5 Ablation experiments on CICIoT2023 dataset

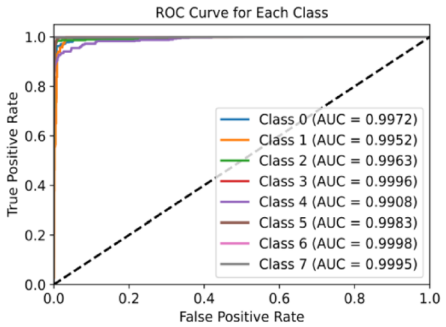
Model	ACC	Precision	Recall	F1-score	Time (s)
CAE-WA	0.9030	0.9024	0.9030	0.9017	306.75
TCN-WA	0.9582	0.9604	0.9582	0.9585	364.65
CAE-TCN-Cross Entropy-WA	0.9632	0.9653	0.9632	0.9634	1153.31
CAE-TCN-Label Smoothing-WA	0.9628	0.9648	0.9628	0.9629	1782.68
CTWA	0.9643	0.9659	0.9643	0.9645	789.58



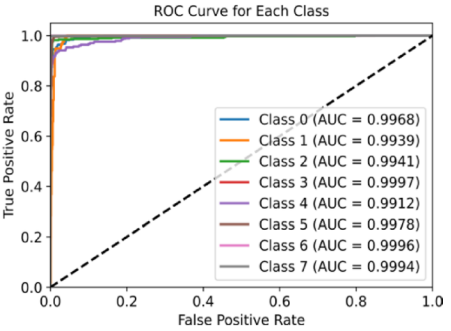
(a) CAE-WA



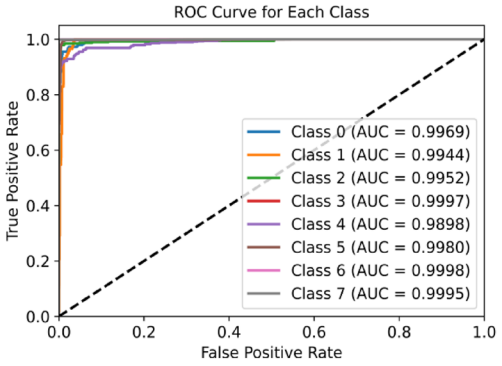
(b) TCN-WA



(c) CAE-TCN-Cross Entropyg-WA

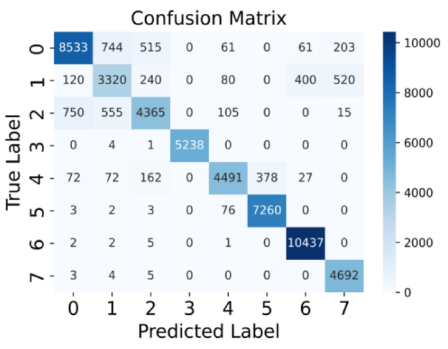


(d) CAE-TCN-Label Smoothing-WA

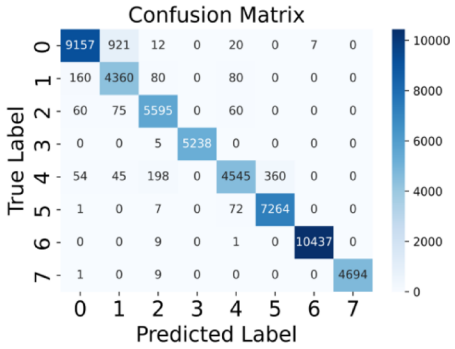


(e) CTWA

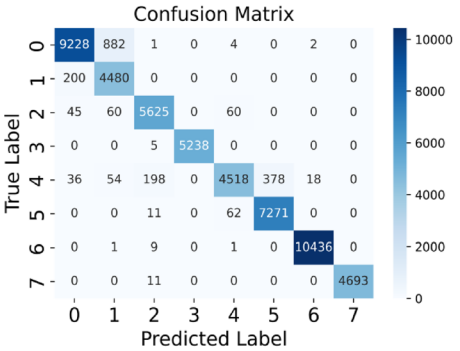
Fig. 7 ROC curve of ablation experiment



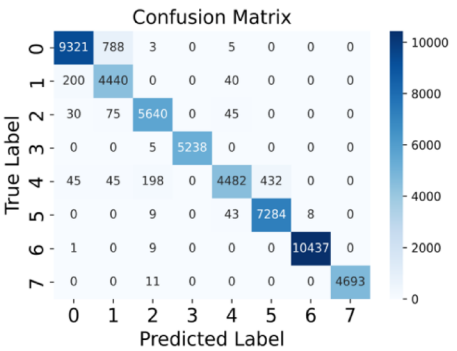
(a) CAE-WA



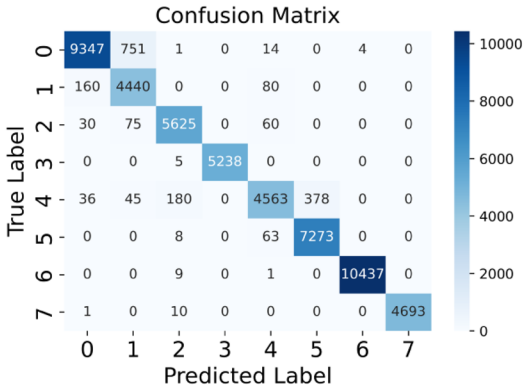
(b) TCN-WA



(c) CAE-TCN-CrossEntropy-WA



(d) CAE-TCN-LabelSmoothing-WA



(e) CTWA

Fig. 8 Confusion matrix diagram of ablation experiment

Table 6 Comparative experiments on the CICIoT2023 dataset

Model	ACC	Precision	Recall	F1-score	Time (s)
SSAE-SOINN	0.8790	0.8805	0.8790	0.8736	1234.18
BDPT	0.9286	0.9299	0.9286	0.9286	282.14
SOINN	0.8393	0.8486	0.8393	0.8182	384.16
IPCA-CNN	0.6608	0.5220	0.6608	0.5656	487.38
NIDS	0.9173	0.9208	0.9173	0.9174	343.27
CTWA	0.9643	0.9659	0.9643	0.9645	789.58

higher than SOINN, 30.35% points higher than IPCA-CNN, and 4.70% points higher than NIDS. Additionally, the time metrics outperform SSAE-SOINN.

The comparative experimental ROC and confusion matrices are shown in Figs. 9 and 10.

The results of this paper's model in using the CICIoT2023 dataset for each category of tag detection are shown in Table 7.

The results of this paper's model in using the BoTNetIoT dataset for each category of tag detection are shown in Table 8.

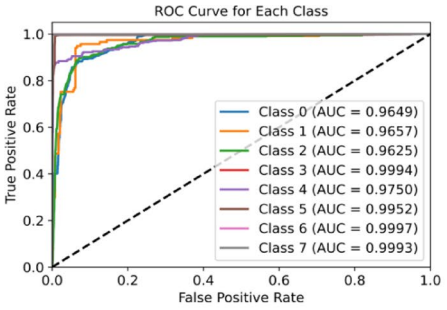
4.6 Analysis of results

The accuracy and loss rate of the model proposed in this paper change with the increasing number of iterations in the dataset, as illustrated in Fig. 11. It can be observed that when the number of iterations in the CICIoT2023 dataset reaches 100, the accuracy achieves 0.9643, while the loss rate falls below 0.05. This indicates a higher degree of stability in the values, suggesting that the model has effectively converged and is capable of producing reliable and consistent results.

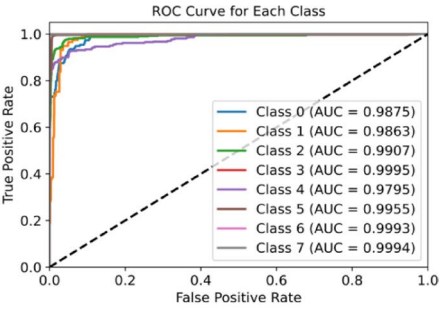
5 Conclusion

In recent years, there has been extensive research on network security issues in the field of the Internet of Things (IoT), and deep learning has been widely applied to intrusion detection in the IoT. This paper proposes an incremental learning framework based on CAE-TCN-WA, which extracts spatiotemporal features through convolutional autoencoders and temporal convolutional networks, and uses a loss function combining cross entropy loss and label balancing loss to improve model performance. Optimized the parameters of the newly added classification layer in the extended model to address issues such as class imbalance and overfitting. In the incremental learning phase, task-specific layers are responsible for sharing underlying feature representations. By using Gaussian distribution to determine whether a sample belongs to a new task, the purpose of distinguishing between old and new tasks can be achieved. Applying WA technology between task heads can adapt to new categories without affecting the performance of the initial task. In the end, the accuracy of the model reached 0.9643 on CICIoT2023 dataset, which can detect unknown attacks. The achievements made are of great significance in the field of network security.

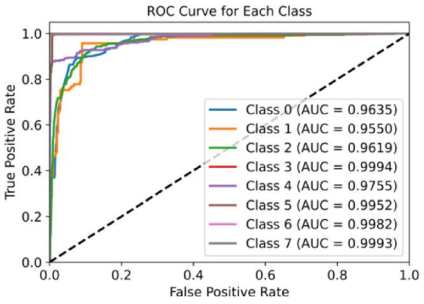
In this paper, we conducted experiments using a balanced set of unknown attack samples. However, such a balanced sample distribution may not fully reflect the real-world situation of attack samples, especially for unknown attacks with small sample sizes. Therefore, the detection performance of the model may not be satisfactory when dealing with unknown



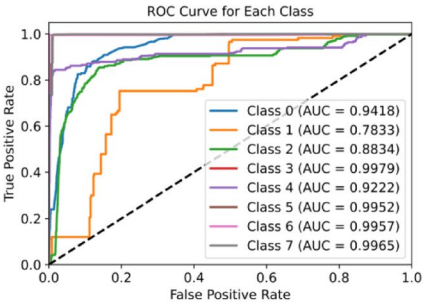
(a) SSAE-SOINN



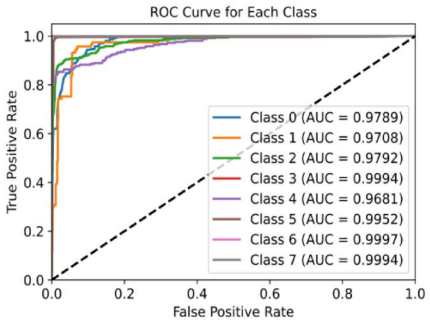
(b) BDPT



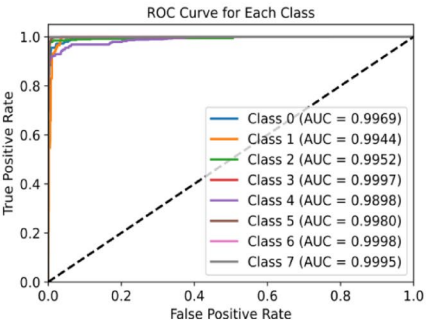
(c) SOINN



(d) IPCA-CNN



(e) NIDS



(f) CTWA

Fig. 9 ROC curve of comparison experiment

attacks with small sample sizes. Additionally, during the training process, we used a fixed set of hyperparameters. Although these hyperparameters performed well in the current experiments, they may not be the optimal choice, especially when facing datasets of different scales or distributions. Moreover, the model structure adopted in this paper is relatively complex, which helps the model learn rich features on large-scale datasets but may also

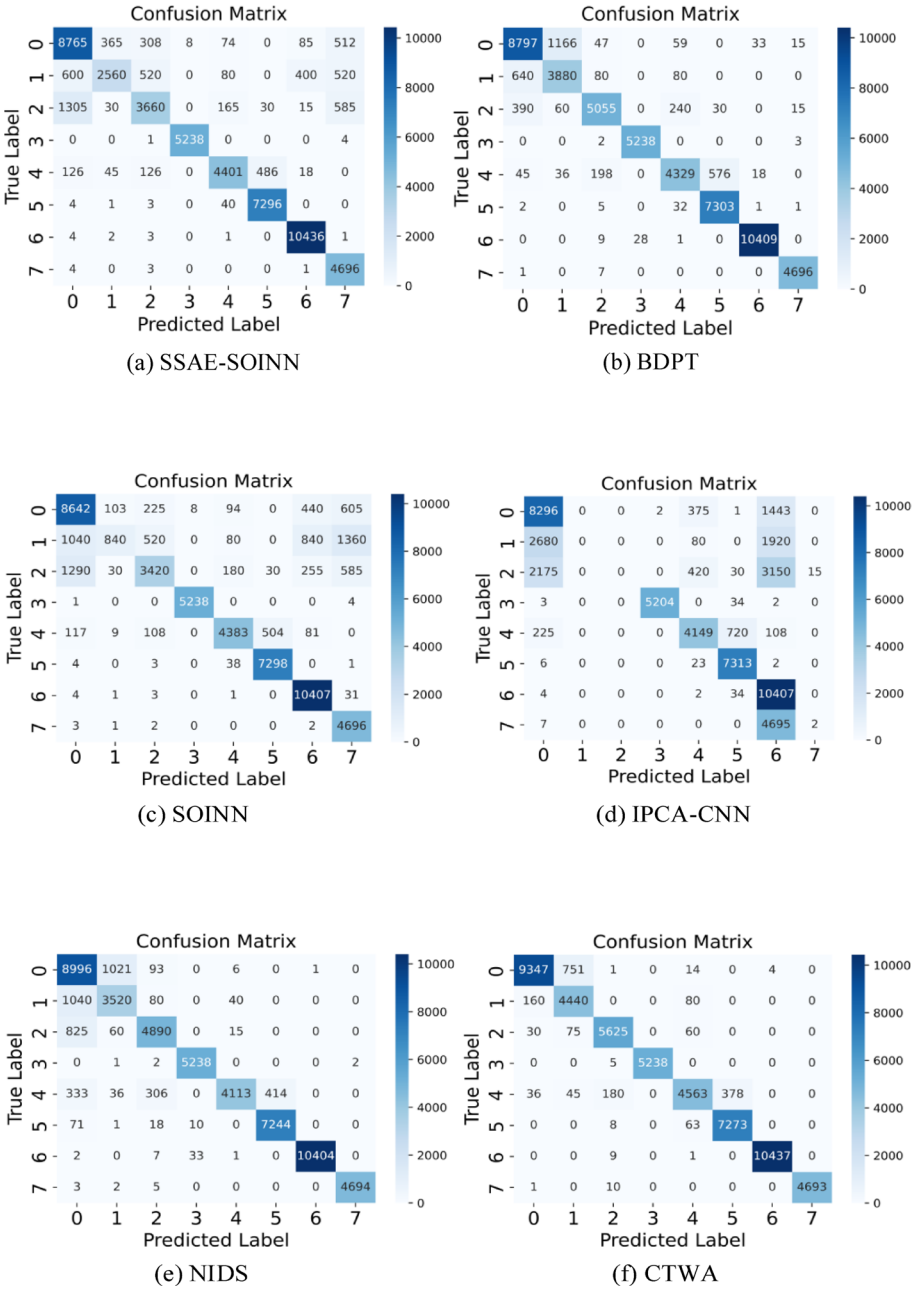


Fig. 10 Comparison experiment confusion matrix diagram

Table 7 Experimental results of various categories in the CICIoT2023 dataset

Tab	Typology	ACC	Precision	Recall	F1-score
0	Benign	0.9239	0.9763	0.9239	0.9494
1	DDoS-ACK	0.9487	0.8360	0.9487	0.8888
2	DNS	0.9715	0.9635	0.9715	0.9675
3	DDoS-ICMP	0.9990	1.0000	0.9990	0.9995
4	DDoS-HTTP	0.8772	0.9544	0.8772	0.9142
5	DDoS-SYN	0.9506	0.9903	0.9506	0.9701
6	DDoS-TCP	0.9990	0.9996	0.9990	0.9993
7	DDoS-UDP	0.9977	1.0000	0.9977	0.9988

Table 8 Experimental results of various categories in the BoTNetIoT dataset

Tab	Typology	ACC	Precision	Recall	F1-score
0	ack	0.9985	0.7471	0.9985	0.8547
1	combo	0.9999	0.9999	0.9999	0.9999
2	junk	0.9999	0.9999	0.9999	0.9999
3	scan	0.9999	0.9999	0.9999	0.9999
4	syn	0.9999	0.9999	0.9999	0.9999
5	TCP	0.9999	0.9999	0.9999	0.9999
6	UDP	0.6574	0.9977	0.6574	0.7926
7	UDPPLAIN	0.9999	0.9999	0.9999	0.9999

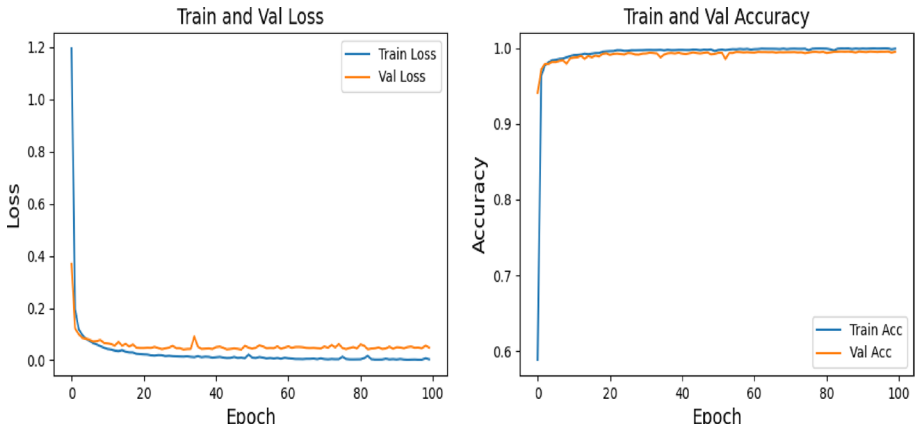


Fig. 11 Loss rate and accuracy rate based on CICIoT2023 dataset

significantly increase the training time. Future work will consider introducing a dynamic hyperparameter adjustment mechanism and exploring more efficient model architectures to improve the model's performance in detecting unknown attacks with small sample sizes and optimize training efficiency.

Author contributions Haizhen Wang: Conceptualization, Formal analysis, Funding acquisition, Methodology, Writing – review & editing. Yutong Yang: Conceptualization, Data curation, Visualization, Validation, Writing – original draft, Writing – review & editing. Pan Tan: Conceptualization, Formal analysis, Validation.

Funding The work is supported by the Fundamental Research Funds Special Project for Research Innovation Platform of Higher Education Institutions of Heilongjiang Province(145409442).

Data availability The data supporting Figs. 1–11 and Tables 1–8 are publicly available in the figshare repository, as part of this record: https://doi.org/10.6084/m9.figshare.30067930.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

Ashraf A, Aziz A, Zahoora U, Rajarajan M, Khan A (2019) Ransomware analysis using feature engineering and deep neural networks. arXiv Preprint. https://doi.org/10.48550/arXiv.1910.00286

Dong J et al (2022) Federated class-incremental learning. In: Proceedings of the IEEE Computer Society conference on computer vision and pattern recognition, pp 10154–10163. https://doi.org/10.48550/arXiv.2203.11473

Hu Z, Li Y, Lyu J, Gao D, Vasconcelos N (2023) Dense network expansion for class incremental learning. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 11858–11867. https://doi.org/10.48550/arXiv.2303.12696

Huang L, Zeng Y, Yang C, An Z, Diao B, Xu Y (2023) eTag: class-incremental learning with embedding distillation and task-oriented generation. arXiv Preprint. https://doi.org/10.48550/arXiv.2304.10103

Jodelet Q, Liu X, Phua YJ, Murata T (2023) Class-incremental learning using diffusion model for distillation and replay. In: Proceedings of the IEEE/CVF international conference on computer vision, pp 3425–3433. https://doi.org/10.48550/arXiv.2306.17560

Kirkpatrick J, Pascanu R, Rabinowitz N et al (2017) Overcoming catastrophic forgetting in neural networks. Proc Natl Acad Sci USA 114(13):3521–3526. https://doi.org/10.1073/pnas.1611835114

Koroniotis N et al (2019) Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: bot-iot dataset. Future Generation Comput Syst 100:779–796. https://doi.org/10.1016/j.future.2019.05.041

Li Z, Hoiem D (2017) Learning without forgetting. IEEE Trans Pattern Anal Mach Intell 40(12):2935–2947. https://doi.org/10.1109/TPAMI.2017.2773081

Liu Y et al (2021) RMM: reinforced memory management for class-incremental learning. In: Proceedings of the conference on neural information processing systems, pp 3478–3490

Liu Q, Zhang Y, Zhou W et al (2023) Adaptive class incremental learning for Internet of Things intrusion detection system. Comput Eng 49(2):169–174. https://doi.org/10.19678/j.issn.1000-3428.0063917(in Chinese)

Lopes N, Ribeiro B, Lopes N, Ribeiro B (2015) Deep belief networks (DBNs). Mach Learn Adapt Many-Core Machines-A Practical Approach 155–186. https://doi.org/10.1007/978-3-319-06938-8_8

Lopez-Paz D, Ranzato MA (2017) Gradient episodic memory for continual learning. In: Advances in neural information processing systems, vol 30

Neto ECP et al (2023) CICIoT2023: a real-time dataset and benchmark for large-scale attacks in IoT environment. Sensors 23(13):5941. https://doi.org/10.3390/s23135941

Petit G et al (2023) FeTrIL: feature translation for exemplar-free class-incremental learning. In: Proceedings of the IEEE/CVF winter conference on applications of computer vision, pp 3900–3909. https://doi.org/10.48550/arXiv.2211.13131

Rebuffi S-A, Kolesnikov A, Sperl G, Lampert CH (2017) iCaRL: incremental classifier and representation learning. In: Proceedings of the IEEE conference on computer vision and pattern recognition. https://doi.org/10.48550/arXiv.1611.07725

Shin H, Lee JK, Kim J et al (2017) Continuous learning with deep generative replay. In: Proceedings of the annual conference on neural information processing systems. Long Beach, USA, pp 2990–2999

Tabassum A, Erbad A, Mohamed A, Guizani M (2021) Privacy-preserving distributed IDS using incremental learning for IoT health systems. IEEE Access 9:14271–14283. https://doi.org/10.1109/ACCESS.2021.3051530

Tian S et al (2023) Continuous transfer of neural network representational similarity for incremental learning. Neurocomputing 545:126300

Van De Ven GM, Li Z, Tolias AS (2021) Class-incremental learning with generative classifiers. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 3611–3620. https://doi.org/10.48550/arXiv.2104.10093

Wang FY, Zhou DW, Ye HJ, Zhan DC (2022) FOSTER: feature boosting and compression for class-incremental learning. In: Avidan S, Brostow G, Cissé M, Farinella GM, Hassner T (eds) Computer Vision – ECCV 2022. Lecture Notes in Computer Science, vol 13685. Springer, Cham. https://doi.org/10.1007/978-3-031-19806-9_23.

Wang WJ, Zhang CY, Wang LY et al (2023) An incremental intrusion detection model fusing IPCA and CNN. J North China Univ Sci Technol (Natural Sci Edition) 45(3):90–100 (in Chinese)

Wen B, Zhu Q (2023) Class-Incremental learning based on big dataset Pre-Trained models. IEEE Access 11:62028–62038

Wu Y, Chen Y, Wang L, Ye Y, Liu Z, Guo Y, Fu Y (2019) Large scale incremental learning. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 374–382

Wu Z et al (2023) Research on network anomaly detection based on SOINN incremental self-encoder. J Ordnance Equip Eng 44(5) **(in Chinese)**

Xu C et al (2018) An intrusion detection system using a deep neural network with gated recurrent units. IEEE Access 6:48697–48707. https://doi.org/10.1109/ACCESS.2018.2867564

Yan S, Xie J, He X (2021) DER: dynamically expandable representation for class incremental learning. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. https://doi.org/10.48550/arXiv.2103.16788

Yoon J, Yang E, Lee J, Hwang SJ (2017) Lifelong learning with dynamically expandable networks. arxiv preprint. https://doi.org/10.48550/arXiv.1708.01547

Zahoora U, Khan A, Rajarajan M et al (2022) Ransomware detection using deep learning based unsupervised feature extraction and a cost sensitive Pareto ensemble classifier. Sci Rep 12:15647. https://doi.org/10.1038/s41598-022-19443-7

Zhang L et al (2023) Class-Incremental learning based on anomaly detection. IEEE Access 11:69423–69438. https://doi.org/10.1109/ACCESS.2023.3293524

Zhao P et al (2022) Intrusion detection model using temporal convolutional network blend into attention mechanism. Int J Inform Secur Priv 16(1):1–20. https://doi.org/10.4018/IJISP.290832

Zhu F et al (2021a) Class-incremental learning via dual augmentation. In: Advances in neural information processing systems, vol 34, pp 14306–14318

Zhu F, Zhang X-Y, Wang C, Yin F, Liu C-L (2021b) Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR), pp 5871–5880

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.